

- 哈希
 - 两数之和
 - 字母异位词分组
 - 最长连续序列
- 双指针
 - 移动零
 - 盛最多水的容器
 - 三数之和
 - 接雨水
- 滑动窗口
 - 无重复字符的最长子串
 - 找到字符串中所有字母异位词
- 子串
 - 和为 K 的子数组
 - 滑动窗口最大值
 - 最小覆盖子串
- 普通数组
 - 最大子数组和
 - 合并区间
 - 轮转数组
 - 除自身以外数组的乘积
 - 缺失的第一个正数
- 链表
 - 相交链表
 - 反转链表
 - 回文链表
 - 环形链表
 - 环形链表 II
 - 合并两个有序链表
 - 两数相加
 - 删除链表的倒数第 N 个结点
 - 两两交换链表中的节点
 - K 个一组翻转链表
 - 随机链表的复制
 - 排序链表
 - 合并 K 个升序链表
 - LRU 缓存

- 二叉树
 - 二叉树的中序遍历
 - 二叉树的最大深度
 - 翻转二叉树
 - 对称二叉树
 - 二叉树的直径
 - 二叉树的层序遍历
 - 将有序数组转换为二叉搜索树
 - 验证二叉搜索树
 - 二叉搜索树中第K小的元素
 - 二叉树的右视图
 - 二叉树展开为链表
 - 从前序与中序遍历序列构造二叉树
 - 路径总和 III
 - 二叉树的最近公共祖先
 - 二叉树中的最大路径和
- 回溯
 - 全排列
 - 子集
 - 电话号码的字母组合
 - 组合总和
 - 括号生成
 - 单词搜索
 - 分割回文串
 - N 皇后
- 栈
 - 有效的括号
 - 最小栈
 - 字符串解码
 - 每日温度
 - 柱状图中最大的矩形
- 堆
 - 数组中的第K个最大元素
 - 前 K 个高频元素
 - 数据流的中位数

序号	方法	功能
1	List<> list = Arrays.asList(res)	数组转为列表
2	array = list.toArray(array);	列表转为数组
3	char[] charArray = str.toCharArray();	String转为char[]
4	String str = new String(charArray); String str = String.valueOf(charArray);	char[]转为String
5	String str = sb.toString();	StringBuilder转为String

哈希

两数之和

简单

题目：给定一个整数数组 **nums** 和一个整数目标值 **target**，请你在该数组中找出 **和为目标值 target** 的那 **两个** 整数，并返回它们的**数组下标**。

解：注意是返回下标

方法一：暴力枚举。枚举数组中的每一个数 **x**，寻找数组中是否存在 **target - x**。两层 for 循环， $O(N^2)$ 时间复杂度

方法二：哈希表。对于每一个 **x**，我们首先查询哈希表中是否存在 **target - x**，然后将 **x** 插入到哈希表中。 $O(N)$ 时间复杂度

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<Integer, Integer>();
        for (int i = 0; i < nums.length; ++i) {
            if (map.containsKey(target - nums[i])) {
                return new int[]{map.get(target - nums[i]), i}; // 返回结果
            }
            map.put(nums[i], i); // 插入这个数以及它的下标
        }
        return new int[0];
    }
}
```

字母异位词分组

中等

题目：给你一个字符串数组，请你将 **字母异位词** 组合在一起。可以按任意顺序返回结果列表。**字母异位词** 是由重新排列源单词的所有字母得到的一个新单词。

示例 1:

```
输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
输出: [["bat"],["nat","tan"],["ate","eat","tea"]]
```

示例 2:

```
输入: strs = [""]
输出: [[""]]
```

示例 3:

```
输入: strs = ["a"]
输出: [["a"]]
```

解:

由于互为字母异位词的两个字符串包含的字母相同，因此对两个字符串分别进行排序之后得到的字符串一定是相同的，故可以将排序之后的字符串作为哈希表的键，值则是字符串列表。

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<String, List<String>>();
        for (String str : strs) {
            char[] array = str.toCharArray();//字符串转为字符数组
            Arrays.sort(array);
            String key = new String(array);//字符数组转为字符串
            List<String> list = map.getOrDefault(key, new ArrayList<String>());//常用api要很熟
            list.add(str);
            map.put(key, list);
        }
        return new ArrayList<List<String>>(map.values());//又学到了
```

```
}  
}
```

最长连续序列

中等

题目：给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。要求时间复杂度为 $O(n)$

示例 1:

输入: `nums = [100,4,200,1,3,2]`
输出: 4
解释: 最长数字连续序列是 `[1, 2, 3, 4]`。它的长度为 4。

示例 2:

输入: `nums = [0,3,7,2,5,8,4,6,0,1]`
输出: 9

解:

每个数 x ，考虑以其为起点，不断尝试匹配 $x+1, x+2, \dots$ 是否存在。并且**每次在哈希表中检查是否存在 $x-1$** ，判断是否需要跳过。因为如果 $x-1$ 存在，那么以 x 为起点就必然会短于以 $x-1$ 为起点。

```
class Solution {  
    public int longestConsecutive(int[] nums) {  
        //先将元素全部放入set  
        Set<Integer> numSet = new HashSet<Integer>();  
        for (int num : nums) {  
            numSet.add(num);  
        }  
  
        int res = 0; //初始化  
        for (int num : numSet) {  
            if (!numSet.contains(num - 1)) { //如果num-1不存在  
                int currentNum = num;  
                int currentStreak = 1;  
                while (numSet.contains(currentNum + 1)) { //如果存在currentNum + 1  
                    currentNum ++;  
                }  
            }  
        }  
    }  
}
```

```
        currentStreak ++;
    }
    res = Math.max(res, currentStreak);
}
}
return res;
}
```

双指针

移动零

简单

题目：给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。**请注意**，必须在不复制数组的情况下原地对数组进行操作。

解：

交换，需要注意的是 `j` 也要往后移。解释：例如 `nums=[1, 2, 0]`，只有 `j` 跟着移动了，才不会出错。

```
class Solution {
    public void moveZeroes(int[] nums) {
        //双指针,i用于遍历,j是下一个非零元素应该在的位置
        int j=0;
        for(int i=0;i<nums.length;i++){
            //有个缺点，如果本来就“有序”，那么依然会进行交换操作
            if(nums[i]!=0){
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
                j++;
            }
        }
    }
}
```

对“有序”依然交换的问题进行优化，可以加一个判断，但leetcode测试反而更慢了，感觉是测试用例的原因。

```
if(nums[i]!=0){
    if(i!=j){//加个判断反而更慢了，推测与测试用例有关
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
    j++;
}
```

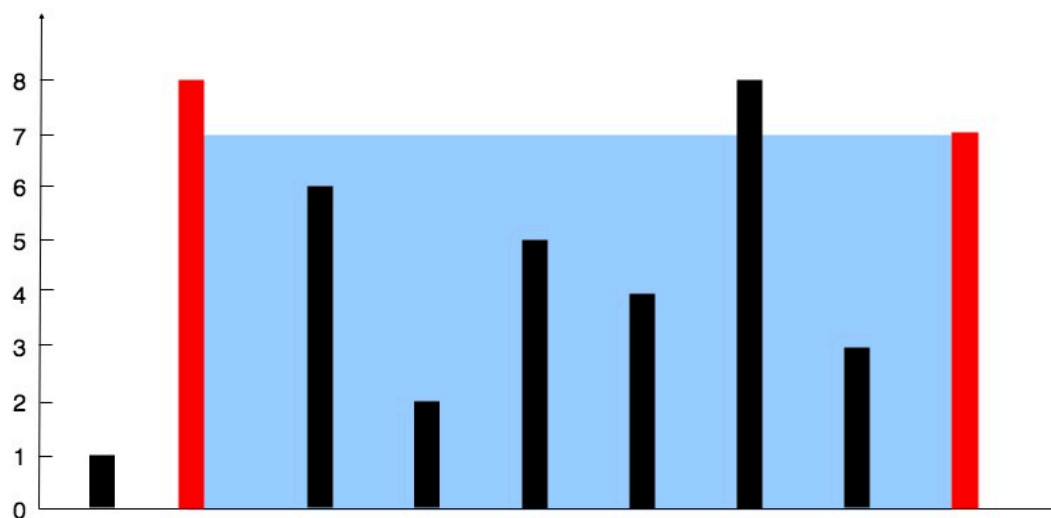
盛最多水的容器

中等

题目：给定一个长度为 n 的整数数组 $height$ 。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, height[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。返回容器可以储存的最大水量。

示例 1：



输入：[1,8,6,2,5,4,8,3,7]

输出：49

解释：图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

解：

也就是矩形面积最大，矩形的高是 $\text{Math.min}(\text{left}, \text{right})$ ，双指针往中间缩就行

```

class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1; // 初始位置
        int ans = 0;
        while (left < right) { // 没有意义
            int area = Math.min(height[left], height[right]) * (right - left);
            ans = Math.max(ans, area);
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return ans;
    }
}

```

三数之和

中等

题目：给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。****注意：答案中不可以包含重复的三元组。**

解：

和两数之和类似，是非常经典的面试题，但是做法不尽相同。简单地使用三重循环枚举所有的三元组，然后哈希表去重复杂度会很高。改进后的伪代码如下。为什么可以使用双指针，因为 $a+b+c=0$ ，当 a 固定时， b 和 c 是此消彼长的关系，那就可以从两端逼近。总时间复杂度为 $O(N^2)$

```

nums.sort()
for first = 0 .. n-1
    // 只有和上一次枚举的元素不相同，我们才会进行枚举
    if first == 0 or nums[first] != nums[first-1] then
        // 第三重循环对应的指针
        third = n-1
        for second = first+1 .. n-1
            if second == first+1 or nums[second] != nums[second-1] then
                // 向左移动指针，直到 a+b+c 不大于 0
                while nums[first] + nums[second] + nums[third] > 0
                    third = third-1
                // 判断是否有 a+b+c==0

```



```
check(first, second, third)
```

作者：力扣官方题解

java代码

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        int n = nums.length;
        Arrays.sort(nums);
        List<List<Integer>> ans = new ArrayList<List<Integer>>();
        // 枚举 a
        for (int first = 0; first < n; first++) {
            // 需要和上一次枚举的数不相同
            if (first > 0 && nums[first] == nums[first - 1]) {
                continue;
            }
            // c 对应的指针初始指向数组的最右端
            int third = n - 1;
            int target = -nums[first];
            // 枚举 b
            for (int second = first + 1; second < n; second++) {
                // 需要和上一次枚举的数不相同
                if (second > first + 1 && nums[second] == nums[second - 1]) {
                    continue;
                }
                // 需要保证 b 的指针在 c 的指针的左侧
                while (second < third && nums[second] + nums[third] > target) {
                    third--;
                }
                // 如果指针重合，随着 b 后续的增加
                // 就不会有满足 a+b+c=0 并且 b<c 的 c 了，可以退出循环
                if (second == third) {
                    break;
                }
                if (nums[second] + nums[third] == target) {
                    List<Integer> list = new ArrayList<Integer>();
                    list.add(nums[first]);
                    list.add(nums[second]);
                    list.add(nums[third]);
                    ans.add(list);
                }
            }
        }
        return ans;
    }
}
```

接雨水

困难-代码简单

题目：给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1:



输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

解:

对于每一个柱子接的水，那么它能接的水= $\min(\text{左右两边最高柱子}) - \text{当前柱子高度}$ 。

```
class Solution {
    public int trap(int[] height) {
        int ans = 0, left = 0, right = height.length - 1, preMax = 0, sufMax = 0;
        while (left < right) {
            preMax = Math.max(preMax, height[left]); // 左边最高柱子
            sufMax = Math.max(sufMax, height[right]); // 右边最高柱子
            // 把区域分成三块，分别是preMax，中间（left到right这个范围），sufMax。
            // 如果preMax < sufMax那么left这一列左右柱子最小值就是preMax，就可以计算left
            // 这列可以接的雨水
            ans += preMax < sufMax ? preMax - height[left++] : sufMax - height[right--];
        }
        return ans;
    }
}
```

//灵茶山艾府视频

滑动窗口

模板:

```
//外层循环扩展右边界，内层循环扩展左边界
for (int l = 0, r = 0 ; r < n ; r++) {
    //当前考虑的元素
    while (l <= r && check()) { //区间[left,right]不符合题意
        //扩展左边界
    }
    //区间[left,right]符合题意，统计相关信息
}
```

无重复字符的最长子串

中等

题目：给定一个字符串 **s**，请你找出其中不含有重复字符的 **最长子串** 的长度。

解：

要熟悉滑动窗口的写法。不含有重复字符——>set。滑动窗口也是双指针，考虑极端情况：全是不重复的字符，那么right会指向最后一个字符，但是这个过程是一步一步移过去的，移动过程中，去重，移动 left。

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        //滑动窗口
        char[] ss = s.toCharArray();
        Set<Character> set = new HashSet<>(); //去重
        int res = 0; //结果
        for(int left = 0, right = 0; right < s.length(); right++) { //每一轮右端点都
            //扩一个。
            char ch = ss[right]; //right指向的元素，也是当前要考虑的元素
            //为什么要用while? 答：例如abc d efd，如果想把后面的d加进子串，那么就要把前面的abcd都删了
            while(set.contains(ch)) { //set中有ch，则缩短左边界，同时从set集合中移除元素
                set.remove(ss[left]);
                left++;
            }
            set.add(ss[right]); //别忘了。将当前元素加入。
            res = Math.max(res, right - left + 1); //计算当前不重复子串的长度。
        }
        return res;
    }
}
//看Solution文件，那里有更好的map解法
```

找到字符串中所有字母异位词

中等

题目：给定两个字符串 **s** 和 **p**，找到 **s** 中所有 **p** 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。**异位词** 指由相同字母重排列形成的字符串（包括相同的字符串）。

示例 1:

输入：s = "cbaebabacd", p = "abc"

输出：[0,6]

解释：

起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。

解：

排列——> **串中每个字符的个数都一样，**因为题目中串只包含小写字母，所以可以用一个长度为26的数组来存储字符串中每个字符的个数。Arrays.equals(), 又学到了。

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int n = s.length(), m = p.length();
        List<Integer> res = new ArrayList<>();
        if(n < m) return res; //如果p比s都长，直接返回
        int[] pCnt = new int[26];
        int[] sCnt = new int[26];
        for(int i = 0; i < m; i++){ // 注意<m
            pCnt[p.charAt(i) - 'a']++;
            sCnt[s.charAt(i) - 'a']++;
        }
        if(Arrays.equals(sCnt, pCnt)){ //一开始是否就是异位词
            res.add(0);
        }
        //例如 n=10, m=3。开始滑动
        for(int i = 0; i < n-m; i++){ //注意
            sCnt[s.charAt(i) - 'a']--; //去掉左边的
            sCnt[s.charAt(i+m) - 'a']++; //加上右边的
            if(Arrays.equals(sCnt, pCnt)){
                res.add(i + 1);
            }
        }
        return res;
    }
}
```

子串

和为 K 的子数组

中等

题目：给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。子数组是数组中元素的**连续非空序列**。

解：

“前缀和”：对于数组中的位置 `j`，前缀和 `pre[j]` 是数组中从第一个元素到第 `j` 个元素的总和。将问题转化为求解**两个前缀和之差**等于`k`的情况。

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        //key: 前缀和    value: 该前缀和的数量
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1); // 空子数组时，前缀和为0，此时计数为1
        int preSum = 0;
        int res = 0;
        for (int i = 1; i <= nums.length; i++) {
            preSum += nums[i - 1];
            //前缀和之差: preSum - (preSum - k) = k
            if(map.containsKey(preSum - k)) res+=map.get(preSum - k);
            map.put(preSum, map.getOrDefault(preSum, 0) + 1);
        }
        return res;
    }
}
```

滑动窗口最大值

困难

题目：给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。返回 **滑动窗口中的最大值**

示例 1：

输入: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

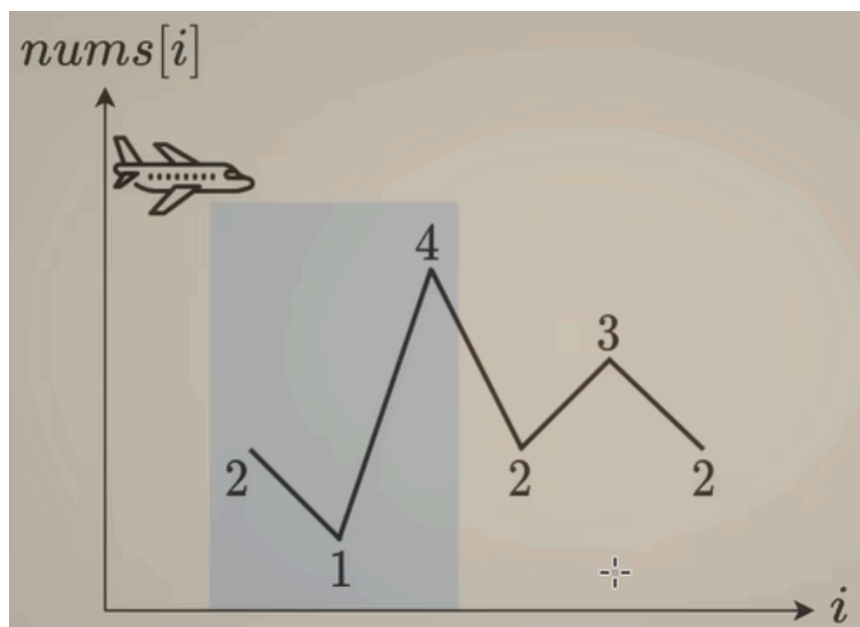
输出: `[3,3,5,5,6,7]`

解释:

滑动窗口的位置	最大值
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

解:

使用**优先队列**，从队首到队尾递减，滑动的时候，遇到比队尾大的值，就while删除队尾那些小的值，并把这个值加入队列。如果队首元素不在窗口内了，要把它移除。



维护一个优先队列: `[4]`——>`[4,2]`——>`[4,3]`——>`[3,2]`，最大值都在队首

注意，这个`[4]`并不是一开始就得到的，而是`[2]`——>`[2,1]`——>`[4]`。

更要注意的是，队列存储的是下标，不是值!!! 因为需要下标去判断这个元素还在不在窗口之内

这些API也要熟悉

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] ans = new int[n - k + 1]; // 答案数组
        Deque<Integer> q = new ArrayDeque<>(); // 双端队列，存储的是下标
        for (int i = 0; i < n; i++) {
```

```

// 1. 入
while (!q.isEmpty() && nums[q.getLast()] <= nums[i]) {
    q.removeLast(); // 维护 q 的单调性
}
q.addLast(i);

// 2. //队首如果超出窗口范围了
if (i - q.getFirst() >= k) { //注意>=
    q.removeFirst();
}
// 3. 首次记录答案时，需要将窗口里面的元素都遍历完了，再记录答案
if (i >= k - 1) {
    // 由于队首到队尾单调递减，所以窗口最大值就是队首
    ans[i - k + 1] = nums[q.getFirst()];
}
}
return ans;
}
}

```

最小覆盖子串

困难

题目：给你一个字符串 **s**、一个字符串 **t**。返回 **s** 中涵盖 **t** 所有字符的最小子串。如果 **s** 中不存在涵盖 **t** 所有字符的子串，则返回空字符串 ""。

示例 1:

输入: s = "ADOBECODEBANC", t = "ABC"
 输出: "BANC"
 解释: 最小覆盖子串 "BANC" 包含来自字符串 t 的 'A'、'B' 和 'C'。

解:

枚举 s 子串的右端点 right，如果子串涵盖 t，就不断移动左端点 left 直到不涵盖为止。代码长，但是逻辑不难。

```

class Solution {
    public String minWindow(String S, String t) {
        char[] s = S.toCharArray();
        int n = s.length;
        //大写字母65-90, 小写字母97-122
        int[] sArr = new int[123], tArr = new int[123];
        int ansLeft=-1, ansRight=n, l=0;
    }
}

```

```

//统计t中每种字符的数量,也可以理解为t比窗口中多的字符种类数
int charNum = 0;
for(char a : t.toCharArray()){
    if(tArr[a]==0) charNum++; //字符直接当作数字使用
    tArr[a]++;
}

//右端点逐渐往右移
for(int r=0;r<n;r++){
    char ch = s[r];
    sArr[ch]++; //加入右端点处字符
    if(sArr[ch]==tArr[ch]){
        charNum--;
    }
    while(charNum==0){ //注意是while
        if(r-l<ansRight-ansLeft){ //找到了更短的子串
            ansLeft=l;
            ansRight=r;
        }
        char b = s[l++]; //取出窗口最左边的字符,缩小窗口
        if(sArr[b]-- == tArr[b]){ //即使现在相等,但移除之后必然就小于t中的
            charNum++;
        }
    }
}
return ansLeft>=0 ? S.substring(ansLeft,ansRight+1) : "";
}
}

```

普通数组

最大子数组和

中等

题目：给你一个整数数组 **nums**，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。**子数组**：数组中的一个连续部分。

解：

核心： **$pre = \text{Math.max}(pre + x, x)$** ；如果前边累加后还不如自己本身大，那就把前边的都扔掉，从此自己本身重新开始累加。

```

class Solution {
    public int maxSubArray(int[] nums) {
        int count = 0, res = nums[0];
        for(int num:nums){

```



```

        count=Math.max(count+num,num);//即count>0 才累加，否则从头开始
        res = Math.max(res, count);// 更新最大和
    }
    return res;
}
}

```

合并区间

中等

题目：以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例 1:

输入: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
 输出: `[[1,6],[8,10],[15,18]]`
 解释: 区间 `[1,3]` 和 `[2,6]` 重叠，将它们合并为 `[1,6]`。

解：先按照区间左边界进行排序，这样对于两个区间，只需要比较`left2`和`right1`即可：

1. 当 `right1` 大于 `left2` 的时候，这两个区间就可以合并了。
2. 但如果发现 `right1` 小于 `left2`，则区间就无法继续合并，就需要开辟一个新的区间来继续执行合并的流程。

重点掌握`Arrays.sort()`如何写，以及列表如何转为数组，`list.toArray()`。

```

class Solution {
    public int[][] merge(int[][] intervals) {
        //Lambda表达式，该函数接受两个int[]类型的参数（interval1和interval2）
        Arrays.sort(intervals, (interval1, interval2) -> interval1[0] - interval2[0]);
        List<int[]> merged = new ArrayList<>();
        for (int i = 0; i < intervals.length; i++) {
            int L = intervals[i][0], R = intervals[i][1];
            //新增：刚开始的时候 or 当前数组的L > 列表里面最近添加的数组的R
            if (merged.size() == 0 || L > merged.get(merged.size() - 1)[1]) {
                merged.add(new int[]{L, R});
            } else { //合并：之前R和当前R的较大值
                merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() - 1)[1], R);
            }
        }
    }
}

```

```
    }  
    return merged.toArray(new int[merged.size()][]); //列表转为数组  
}  
}
```

轮转数组

中等

题目：给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

输入: `nums = [1,2,3,4,5,6,7]`, `k = 3`
输出: `[5,6,7,1,2,3,4]`
解释:
向右轮转 1 步: `[7,1,2,3,4,5,6]`
向右轮转 2 步: `[6,7,1,2,3,4,5]`
向右轮转 3 步: `[5,6,7,1,2,3,4]`

解:

轮转 k 次等于轮转 $k \% n$ 次。可以推出公式是: `arr[] = [n-k,n)+[0,n-k)`

方法一：另外开创一个数组，空间复杂度 $O(n)$

```
class Solution {  
    public void rotate(int[] nums, int k) {  
        int n = nums.length;  
        int[] arr = new int[n];  
        k = k % n;  
        for(int i=0;i<k;i++){  
            arr[i] = nums[n-k+i];  
        }  
        for(int i=0;i<n-k;i++){  
            arr[k+i] = nums[i];  
        }  
        for(int i=0;i<n;i++){  
            nums[i] = arr[i];  
        }  
    }  
}
```

方法二：原地做法，空间复杂度 $O(1)$

1	2	3	4	5	6	7
---	---	---	---	---	---	---

$k = 3$

7	6	5	4	3	2	1
---	---	---	---	---	---	---

反转整个数组

5	6	7	4	3	2	1
---	---	---	---	---	---	---

反转前 k 个

5	6	7	1	2	3	4
---	---	---	---	---	---	---

反转后 $n - k$ 个

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n; // 轮转 k 次等于轮转 k%n 次
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }

    private void reverse(int[] nums, int i, int j) {
        while (i < j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            i++;
            j--;
        }
    }
}
```

除自身以外数组的乘积

中等——有难度

题目：给你一个整数数组 `nums`，返回数组 `answer`，其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积。题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。请 ****不要使用除法，****且在 $O(n)$ 时间复杂度内完成此题

示例 1:

输入: `nums = [1,2,3,4]`
输出: `[24,12,8,6]`

解:

容易想到两层for循环,但题目要求 $O(n)$,需要其他方法。使用前后缀分解, `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积。换句话说,如果知道了 `i` 左边所有数的乘积,以及 `i` 右边所有数的乘积,就可以算出 `answer[i]`。于是:

- 定义 `pre[i]` 表示从 `nums[0]` 到 `nums[i-1]` 的乘积。
- 定义 `suf[i]` 表示从 `nums[i+1]` 到 `nums[n-1]` 的乘积。

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] pre = new int[n];
        pre[0] = 1;
        for(int i=1;i<n; i++){
            pre[i] = pre[i-1]*nums[i-1]; //得到pre[0]=1, pre[1]=nums[0],...
        }
        int suf = 1; //后缀数组退化为一个数
        //suf[n-1]=1,suf[n-2]=nums[n-1],...
        //但实际上suf[n-1]不需要,保持原来的pre[n-1]就可以了
        for(int i=n-2;i>=0;i--){ //因此从pre[n-2]开始更新
            suf *= nums[i+1];
            pre[i] *= suf;
        }
        return pre;
    }
}
```

缺失的第一个正数

困难

题目: 给你一个未排序的整数数组 `nums`, 请你找出其中没有出现的最小的正整数。要求时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

示例 1:

输入: `nums = [1,2,0]`
输出: 3

解释：范围 $[1, 2]$ 中的数字都在数组中。

示例 2:

输入: `nums = [3,4,-1,1]`

输出: 2

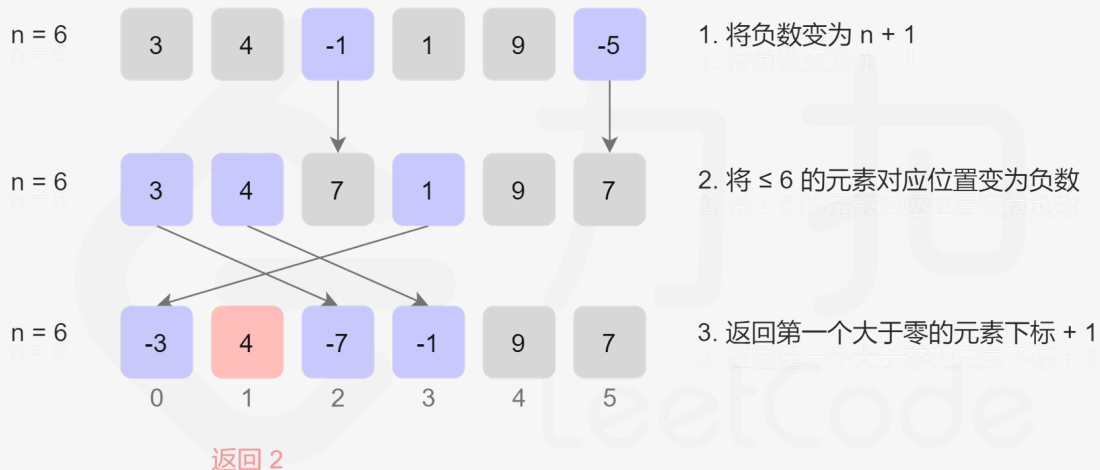
解释: 1 在数组中, 但 2 没有。

解:

如果本题没有额外的时空复杂度要求, 那么就很容易实现。一个重要的结论: 对于一个长度为 N 的数组, 其中没有出现的最小正整数只能在 $[1, N+1]$ 中。因此可以把原数组设计为哈希表。可以先处理掉负数, 再利用负号标记已经出现过的索引位置, 这样, 又不会完全覆盖原来的值。

算法的流程如下:

- 将数组中所有小于等于 0 的数修改为 $N+1$;
- 我们遍历数组中的每一个数 x , 它可能已经被打了标记, 因此原本对应的数为 $|x|$, 其中 $| \cdot |$ 为绝对值符号。如果 $|x| \in [1, N]$, 那么我们给数组中的第 $|x| - 1$ 个位置的数添加一个负号。注意如果它已经有负号, 不需要重复添加;
- 在遍历完成之后, 如果数组中的每一个数都是负数, 那么答案是 $N+1$, 否则答案是第一个正数的位置加 1。



$n=6$ $[-3, 4, -7, -1, 9, 7]$ 3. 返回第一个大于零的元素下标 + 1

返回 2

核心: 用负号表示 $[\text{索引值}+1]$ 这个数是存在的

用负号标识【索引+1】这个数是存在的，没有出现的最小正整数介于[1,n+1]。理解：

1. 对数组排个序，如果1到n都有，那么没有出现的最小正整数就是n+1。
2. 如果1到n有缺失值，那么没有出现的最小正整数会介于[1,n]。综上，介于[1,n+1]

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        // 0和负数变为n+1
        for (int i = 0; i < n; ++i) {
            if (nums[i] <= 0) {
                nums[i] = n + 1;
            }
        }

        for (int i = 0; i < n; i++) {
            int num = Math.abs(nums[i]);
            if (num <= n) { // 对1~n中的数进行标记，注意映射到下标要减去1
                nums[num - 1] = -Math.abs(nums[num - 1]);
            }
        }
        // 其实是对1~n这些数进行遍历
        for (int i = 0; i < n; i++) {
            if (nums[i] > 0) {
                return i + 1; // 缺失的第一个正数
            }
        }
        return n + 1;
    }
}
```

链表

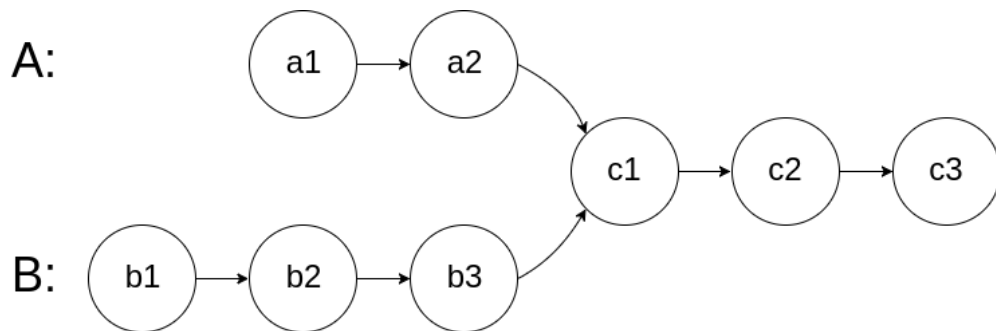
链表的面试题基本都要最优解，不能有额外空间的那种。

相交链表

简单

题目：给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 **null**。

图示两个链表在节点 **c1** 开始相交**： **



题目数据 **保证** 整个链式结构中不存在环。

注意，函数返回结果后，链表必须 **保持其原始结构**。

解：

双指针法。“如果走到尽头也没有找到你，那我就走你走过的路。”

```
public class Solution {  
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {  
        if(headA==null || headB == null) return null;  
        ListNode A = headA, B = headB;  
        while(A!=B){//如果没有交点，第2次都会到达null  
            A = A!=null? A.next: headB;  
            B = B!=null? B.next: headA;  
        }  
        return A;  
    }  
}
```

反转链表

简单

题目：给你单链表的头节点 **head**，请你反转链表，并返回反转后的链表。

解：

方法一：迭代

```
class Solution {  
    public ListNode reverseList(ListNode head) {  
        ListNode cur = head, pre = null;  
        while(cur != null) {  
            ListNode tmp = cur.next; // 暂存后继节点 cur.next  
            cur.next = pre;          // 修改 next 引用指向  
            pre = cur;               // 更新 pre  
            cur = tmp;               // 移动 cur 指针  
        }  
        return pre;  
    }  
}
```

```

        cur = tmp;                // 移到原本的next节点
    }
    return pre;
}
}

```

方法二：递归

递归版本比较复杂。

```

class Solution {
    public ListNode reverseList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }
        //传入下一个节点，开始反转链表的其余部分
        ListNode newHead = reverseList(head.next);
        head.next.next = head; // 将后面一个节点的next指向自己，实现反转
        head.next = null; //将当前节点的next置为null，切断与原链表的联系
        return newHead; // 返回反转后链表的新头节点
    }
}

```

回文链表

简单

题目：给你一个单链表的头节点 **head**，请你判断该链表是否为回文链表。如果是，返回 **true**；否则，返回 **false**。

****进阶：**你能否用 **O(n)** 时间复杂度和 **O(1)** 空间复杂度解决此题？

解：

方法一：将值复制到数组中后用双指针法。

```

class Solution {
    public boolean isPalindrome(ListNode head) {
        List<Integer> list = new ArrayList<>();
        while(head!=null){
            list.add(head.val);
            head = head.next;
        }
        for(int i=0;i<list.size()/2;i++){
            if(list.get(i)!=list.get(list.size()-1-i)) return false;
        }
        return true;
    }
}

```



```
    }  
    return true;  
}  
}
```

方法二：快慢指针。

五个步骤：

1. 找到前半部分链表的尾节点。
2. 反转后半部分链表。
3. 判断是否回文。
4. 恢复链表。
5. 返回结果。

步骤一对于找链表的中点，可以使用快慢指针，一次遍历即可找到。若链表有奇数个节点，则中间的节点应该看作是前半部分。

步骤二可以使用「206. 反转链表」问题中的解决方法来反转链表的后半部分。

步骤三比较两个部分的值，当后半部分到达末尾则比较完成，可以忽略计数情况中的中间节点。

步骤四与步骤二使用的函数相同，再反转一次恢复链表本身。

```
class Solution {  
    public boolean isPalindrome(ListNode head) {  
        // 快慢指针找中点  
        ListNode slow = head, fast = head;  
        while (fast.next != null && fast.next.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
        }  
        ListNode mid = slow;  
        ListNode c = reverse(mid.next); // 反转后半部分  
        ListNode d = head; // 前半部分  
        boolean res = true;  
  
        // 判断  
        while (c != null) {  
            if (c.val != d.val) {  
                res = false;  
                break;  
            }  
            c = c.next; //  
            d = d.next;  
        }  
    }  
}
```

```

        slow.next = reverse(mid.next);//注意！！ 反转并重新建立连接
        return res;
    }

    // 反转链表
    private ListNode reverse(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode temp = curr.next;//必须先暂存
            curr.next = prev;
            prev = curr;
            curr = temp;
        }
        return prev;
    }
}

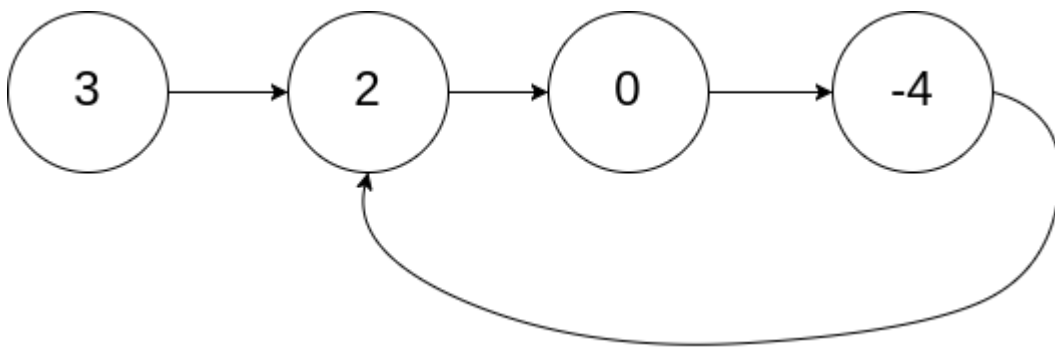
```

环形链表

简单

题目：给你一个链表的头节点 **head**，判断链表中是否有环。如果链表中存在环，则返回 **true**。否则，返回 **false**。

示例 1:



输入: head = [3,2,0,-4], pos = 1
 输出: true
 解释: 链表中有一个环，其尾部连接到第二个节点。

示例 3:



输入: head = [1], pos = -1

输出: false

解释: 链表中没有环。

解:

使用快慢指针。时间复杂度: $O(N)$, 空间复杂度: $O(1)$

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head; // 乌龟和兔子同时从起点出发
        while (fast != null && fast.next != null) { // 注意条件
            slow = slow.next; // 乌龟走一步
            fast = fast.next.next; // 兔子走两步
            if (fast == slow) // 兔子追上乌龟（套圈），说明有环
                return true;
        }
        return false; // 访问到了链表末尾，无环
    }
}
```

环形链表 II

中等

题目: 给定一个链表的头节点 **head**, 返回链表开始入环的第一个节点。 *如果链表无环, 则返回 **null**。*

解:

方法一: 哈希表。一旦遇到了此前遍历过的节点, 就可以判定链表中存在环。

方法二: 快慢指针, 重点掌握。环前面a个节点, 环内b个节点, 根据:

1. $f = 2s$ (快指针每次2步, 路程刚好2倍)
2. $f = s + nb$ (相遇时, 刚好多走了n圈)

推出: $s = nb$

从head结点走到入环点需要走: $a + nb$, 而slow已经走了nb, 那么slow再走a步就是入环点了。

如何知道slow刚好走了a步？从head开始，和slow指针一起走，相遇时刚好就是a步

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        //总路程a+nb
        ListNode fast = head, slow = head;
        //到达nb的地方
        while (true) {
            if (fast == null || fast.next == null) return null;
            fast = fast.next.next;
            slow = slow.next;
            if (fast == slow) break;
        }
        //快指针没用了，重新指向头节点，当fast和slow相遇，就是入口节点
        fast = head;
        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }
        return fast;
    }
}
```

合并两个有序链表

简单

题目：将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

解：

有迭代法和递归法，掌握迭代的就行。

```
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode prehead = new ListNode(-1); //新建一个链表，存储结果
        ListNode prev = prehead;
        while (l1 != null && l2 != null) { //如果两个链表当前位置都不为空
            if (l1.val <= l2.val) { //比较大小
                prev.next = l1;
                l1 = l1.next;
            } else {
                prev.next = l2;
                l2 = l2.next;
            }
            prev = prev.next;
        }
        return prehead.next;
    }
}
```

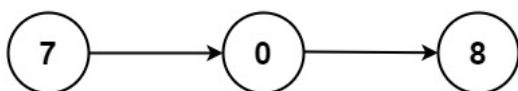
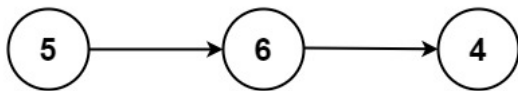
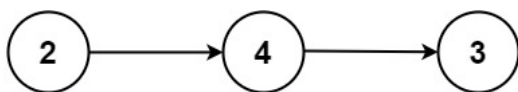
```
}  
// 合并后 l1 和 l2 最多只有一个还未被合并完，我们直接将链表末尾指向未合并完的链表  
即可  
  
prev.next = (l1 == null) ? l2 : l1;  
return prehead.next;  
}  
}
```

两数相加

中等

题目：给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储 **一位** 数字。请你将两个数相加，并以相同形式返回一个表示和的链表。你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

示例 1：



输入：l1 = [2,4,3], l2 = [5,6,4]

输出：[7,0,8]

解释：342 + 465 = 807.

示例 2：

输入：l1 = [0], l2 = [0]

输出：[0]

示例 3：

输入: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
输出: [8,9,9,9,9,0,0,0,1]

注意低位在前，高位在后！！

解：

从低位开始，要考虑进位的问题。此外，还要注意如果最后还有进位，那么需要追加一个节点。

1. 获取值，相加，更新head和tail
2. 计算进位
3. 移动节点
4. 循环结束后如果进位>0，追加一个节点

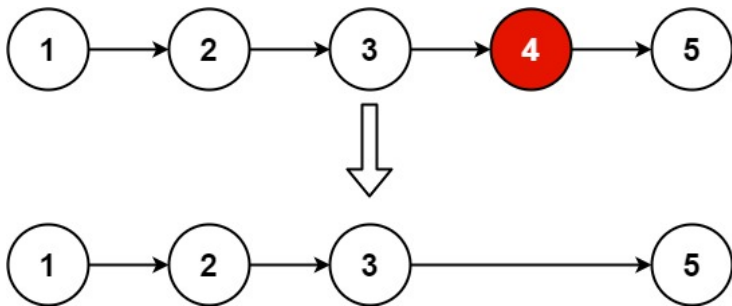
```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode head = null, tail = null;
        int carry = 0; //进位，初值为0
        while (l1 != null || l2 != null) {
            int n1 = (l1 != null) ? l1.val : 0; //关键
            int n2 = (l2 != null) ? l2.val : 0;
            int sum = n1 + n2 + carry;
            if (head == null) { //if只会进入一次，节点值是进位(如果有的话)之后，留下来的值
                head = tail = new ListNode(sum % 10);
            } else { //后续的进位
                tail.next = new ListNode(sum % 10);
                tail = tail.next;
            }
            carry = sum / 10; //进位
            if (l1 != null) {
                l1 = l1.next;
            }
            if (l2 != null) {
                l2 = l2.next;
            }
        }
        //最后，如果还有进位，要再加一个节点
        if (carry > 0) {
            tail.next = new ListNode(carry);
        }
        return head;
    }
}
```

删除链表的倒数第 N 个结点

中等

题目：给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1:



输入: head = [1,2,3,4,5], n = 2
输出: [1,2,3,5]

解:

遍历一遍，得到链表的长度，再找到该节点，设置`cur.next = cur.next.next`。可以在head前创建一个哑节点，这样做可以避免讨论头结点被删除的情况，不管原来的head有没有被删除，直接返回`dummy.next`即可。

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head); // 哑节点值为0，指向头节点
        int length = getLength(head);
        ListNode cur = dummy; // 注意！从哑节点开始
        for (int i = 1; i < length - n + 1; i++) {
            cur = cur.next;
        }
        cur.next = cur.next.next; // 如果只有一个节点且被删，哑节点会指向null
        ListNode ans = dummy.next;
        return ans;
    }

    public int getLength(ListNode head) {
        int length = 0;
        while (head != null) {
            length++;
            head = head.next;
        }
        return length;
    }
}
```

两两交换链表中的节点

中等

题目：给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

解：

递归。1-->2-->3。交换1，2：2.next = 1, 但是1.next不一定是3。对后面的节点需要用递归。反正就记住，递归就是把未知当已知。

终止条件：节点为空或者节点的下一个节点为空。比如说，当只有三个节点时，那么第三个节点.next = null，直接返回它即可。当只有两个节点时，第三个节点本身就是null，让1.next=null也没毛病。

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        if(head==null || head.next==null) return head; //空节点或只有一个节点
        ListNode one = head;
        ListNode two = one.next;
        ListNode three = two.next;

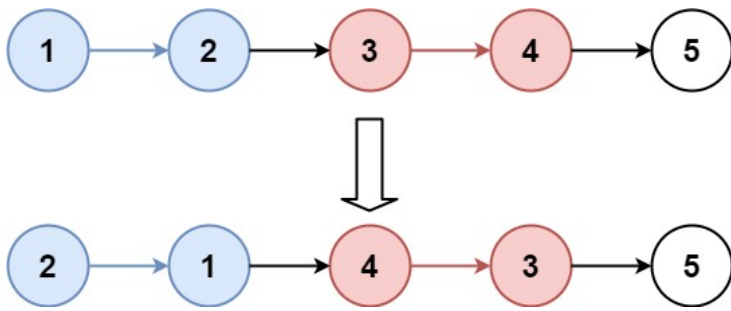
        two.next = one;
        one.next = swapPairs(three); //假设后面的节点已经交换好
        return two;
    }
}
```

K 个一组翻转链表

困难

题目：给你链表的头节点 `head`，每 `k` 个节点一组进行翻转，请你返回修改后的链表。`k` 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 `k` 的整数倍，那么请将最后剩余的节点保持原有顺序。需要实际进行节点交换，要求O(1)空间复杂度。

示例 1：



输入: head = [1,2,3,4,5], k = 2
输出: [2,1,4,3,5]

解:

分组反转，需要用到哑节点，非常需要注意的是，ListNode pre = hair 是引用传递，指向同一个对象，这也是hair的指向为什么可以改变的原因。reverse(): 之前题目《反转链表》是null(pre) -> head -> ... -> null //现在是 head -> ... -> tail -> tail.next //反转之后是tail.next <- head <- ... <- tail //也就是reverse把后面的连接已经弄好了（看箭头，不要看上面的前后位置）

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        //哑节点
        ListNode hair = new ListNode(0);
        hair.next = head;
        ListNode pre = hair; //最开始pre就在hair位置

        while(head != null){
            ListNode tail = pre; //注意
            for(int i=0; i<k; i++){
                tail = tail.next;
                if(tail == null) return hair.next;
            }
            ListNode[] headAndTail = reverse(head, tail);
            head = headAndTail[0];
            tail = headAndTail[1];
            pre.next = head; //前面的连接。第一次时就可以改变hair的指向
            pre = tail;
            head = tail.next;
        }
        return hair.next;
    }

    //反转链表，后面的连接已弄好
    private ListNode[] reverse(ListNode head, ListNode tail){
        ListNode pre = tail.next, cur = head;
        while(pre != tail){
            ListNode temp = cur.next;
            cur.next = pre;
            pre = cur;
            cur = temp;
        }
    }
}
```

```
    }  
    return new ListNode[]{tail, head};  
}  
}
```

随机链表的复制

中等

题目：给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。

构造这个链表的 **深拷贝**。深拷贝应该正好由 n 个 **全新** 节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。**复制链表中的指针都不应指向原链表中的节点。**

例如，如果原链表中有 X 和 Y 两个节点，其中 $X.random \rightarrow Y$ 。那么在复制链表中对应的两个节点 x 和 y ，同样有 $x.random \rightarrow y$ 。

返回复制链表的头节点。

解：

如果是普通链表，可以直接按照遍历的顺序创建链表节点。而本题中因为随机指针的存在，当我们拷贝节点时，「当前节点的随机指针指向的节点」可能还没创建。

递归，使用map存储旧-新节点对，表示这个节点的新节点是否创建好了，如果没有创建，就要递归地去进行创建。

```
class Solution {  
    Map<Node, Node> cachedNode = new HashMap<Node, Node>();  
    public Node copyRandomList(Node head) {  
        if (head == null) return null; //注意  
        if (!cachedNode.containsKey(head)) {  
            Node headNew = new Node(head.val);  
            cachedNode.put(head, headNew); //存储旧-新节点  
            headNew.next = copyRandomList(head.next); //注意  
            headNew.random = copyRandomList(head.random);  
        }  
        return cachedNode.get(head);  
    }  
}
```

排序链表

中等

题目：给你链表的头结点 **head**，请将其按 **升序** 排列并返回 **排序后的链表**。

进阶： $O(n \log n)$ 时间复杂度， $O(1)$ 空间复杂度。

解：

归并排序。注意断开连接后，再对left, right 分别排序，最后合并。

```
class Solution {
    //归并排序
    public ListNode sortList(ListNode head) {
        return sort(head);
    }
    //将节点排序好，并返回头节点
    private ListNode sort(ListNode head){
        if(head==null || head.next==null) return head;//单个节点直接返回
        ListNode mid = findMid(head);//寻找中点
        ListNode l=head, r=mid.next;//前半部分,后半部分的开始
        mid.next = null;//断开连接
        ListNode l2 = sort(l);
        ListNode r2 = sort(r);
        return merge(l2, r2);
    }

    //找中点
    private ListNode findMid(ListNode head){
        ListNode f = head,s=head;
        while(f.next!=null && f.next.next!=null){//注意
            f = f.next.next;
            s = s.next;
        }
        return s;
    }

    //合并两个有序链表
    private ListNode merge(ListNode headA, ListNode headB){
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;//浅拷贝,作用是node可以更新，同时dummy得以保持在原来的位置

        while(headA!=null && headB!=null){
            if(headA.val<headB.val){
                tail.next = headA;
                headA = headA.next;
            }
            else{
                tail.next = headB;
                headB = headB.next;
            }
        }
        if(headA!=null) tail.next = headA;
        if(headB!=null) tail.next = headB;
        return dummy.next;
    }
}
```

```

        }
        tail = tail.next;
    }
    tail.next = (headA!=null)? headA:headB;
    return dummy.next;
}
}

```

合并 K 个升序链表

困难

题目：给你一个链表数组，每个链表都已经按升序排列。请你将所有链表合并到一个升序链表中，返回合并后的链表。

解：

方法一：暴力。按照 [21. 合并两个有序链表](#) 的 [题解思路](#)，先合并前两个链表，再把得到的新链表和第三个链表合并，再和第四个链表合并，依此类推。

方法二：分治法。把 lists 一分为二（尽量均分），先合并前半部分的链表，再合并后半部分的链表，然后把这两个链表合并成最终的链表。如何合并前半部分的链表呢？我们可以继续一分为二。如此分下去直到只有一个链表，此时无需合并。

```

class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        return mergeKLists(lists, 0, lists.length);
    }

    // 递归合并从 lists[i] 到 lists[j-1] 的链表
    private ListNode mergeKLists(ListNode[] lists, int i, int j) {
        int m = j-i;//链表数量
        if(m==0) return null;
        if(m==1) return lists[i];
        ListNode a = mergeKLists(lists,i,i+m/2);//排序前/后半部分
        ListNode b = mergeKLists(lists,i+m/2,j);
        return merge(a,b);
    }

    // 合并两个有序链表
    private ListNode merge(ListNode headA, ListNode headB){
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;//浅拷贝,作用是node可以更新,同时dummy得以保持在原来的位置

        while(headA!=null && headB!=null){
            if(headA.val<headB.val){
                tail.next = headA;
            }
        }
    }
}

```

```

        headA = headA.next;
    }
    else{
        tail.next = headB;
        headB = headB.next;
    }
    tail = tail.next;
}
tail.next = (headA!=null)? headA:headB;
return dummy.next;
}
}

```

LRU 缓存

中等

题目：请你设计并实现一个满足 **LRU (最近最少使用) 缓存** 约束的数据结构。

实现 **LRUCache** 类：

- **LRUCache(int capacity)** 以 **正整数** 作为容量 **capacity** 初始化 LRU 缓存
- **int get(int key)** 如果关键字 **key** 存在于缓存中，则返回关键字的值，否则返回 **-1**。
- **void put(int key, int value)** 如果关键字 **key** 已经存在，则变更其数据值 **value**；如果不存在，则向缓存中插入该组 **key-value**。如果插入操作导致关键字数量超过 **capacity**，则应该 **逐出** 最久未使用的关键字。

函数 **get** 和 **put** 必须以 **O(1)** 的平均时间复杂度运行。

示例：

输入

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

输出

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

解释

```
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // 缓存是 {1=1}
lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}
lRUCache.get(1);    // 返回 1
lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3}
lRUCache.get(2);    // 返回 -1 (未找到)
lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3}
```

```
LRUCache.get(1);    // 返回 -1 (未找到)
LRUCache.get(3);    // 返回 3
LRUCache.get(4);    // 返回 4
```

解：

实现本题的两种操作，需要用到一个哈希表和一个双向链表。在面试中，面试官一般会期望读者能够自己实现一个简单的双向链表，而不是使用语言自带的、封装好的数据结构。

方法一：使用封装好的LinkedHashMap。没怎么看懂

```
class LRUCache extends LinkedHashMap<Integer, Integer>{
    private int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75F, true);
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
        return size() > capacity;
    }
}
```

方法二：哈希表 + 双向链表。

- 双向链表靠近头部的键值对是最近使用的，靠近尾部的键值对是最久未使用的。
- map = {键, 装有值的节点}

在双向链表的实现中，使用一个伪头部（dummy head）和伪尾部（dummy tail）标记界限，这样在添加节点和删除节点的时候就不需要检查相邻的节点是否存在。

```
class LRUCache extends LinkedHashMap<Integer, Integer> {
    // 定义一个链表节点类
    class ListNode {
        int key; // 必须要有key属性，因为删除节点时要同时删除map键值对
        int value;
```

```

        ListNode prev;
        ListNode next;
        public ListNode(){}; //无参构造器
        public ListNode(int key, int value){ //有参构造器
            this.key = key;
            this.value = value;
        }
    }

    // LRU类的一些属性
    private Map<Integer, ListNode> map = new HashMap<>();
    private int capacity; // 容量
    private ListNode head, tail; // 链表头尾

    public LRUCache(int capacity) { // 构造器
        this.capacity = capacity;
        head = new ListNode(); // head和tail都是哑节点
        tail = new ListNode();
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key))
            return -1;
        ListNode node = map.get(key); // 找到节点
        moveToHead(node);
        return node.value;
    }

    public void put(int key, int value) {
        ListNode node = map.get(key);
        if (node == null) { //如果之前没有
            ListNode newNode = new ListNode(key, value);
            map.put(key, newNode);
            addHead(newNode);
            if (map.size() > capacity) {
                ListNode tail = delTail();
                map.remove(tail.key); // 这里就用到key
            }
        }
        else {
            node.value = value;
            moveToHead(node);
        }
    }

    // 4个对节点的操作
    private void moveToHead(ListNode node) { // 移到头部
        delNode(node);
        addHead(node);
    }

    private ListNode delTail() { // 删去尾部节点,有返回值,因为要删去map
        ListNode res = tail.prev;
        delNode(res);
        return res;
    }

```

```

    }

    private void addHead(ListNode node) { // 添加节点到头部
        node.prev = head;
        node.next = head.next;
        head.next.prev = node;
        head.next = node;
    }

    private void delNode(ListNode node) { // 移除一个节点
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
}

```

二叉树

迭代和递归相对应，BFS和DFS相对应。

- 先序，中序和后序都是DFS，可以用递归或者迭代去做。
- 层序的递归实现是DFS，迭代实现是BFS

递归的精髓在于把方法当作已经实现了的，**只关注它的返回值**，即返回值已知。

二叉树的中序遍历

简单

题目：给定一个二叉树的根节点 **root**，返回 它的 **中序** 遍历。

解：

所谓的先中后都是指父节点的遍历顺序。有递归和迭代两种方法。

方法一：递归

```

class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<Integer>();
        inorder(root, res);
        return res;
    }

    public void inorder(TreeNode root, List<Integer> res) {
        if (root == null) {

```



```

        return;
    }
    //左中右
    inorder(root.left, res);
    res.add(root.val);
    inorder(root.right, res);
}
}

```

方法二：

迭代。自己画棵二叉树就很好理解了。

```

class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<Integer>();
        Deque<TreeNode> stack = new LinkedList<TreeNode>();
        while (root != null || !stack.isEmpty()) {
            while (root != null) { //持续向左深入
                stack.push(root);
                root = root.left;
            }
            root = stack.pop();
            res.add(root.val);
            root = root.right;
        }
        return res;
    }
}

```

二叉树的最大深度

简单

题目：给定一个二叉树 **root**，返回其最大深度。二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

解：

也是有递归和迭代解法，也可以叫做深度优先搜索和广度优先搜索

方法一：dfs，后序遍历

```

class Solution {
    public int maxDepth(TreeNode root) {

```

```

        //递归
        if(root==null) return 0;
        int l = maxDepth(root.left);//左边最大深度
        int r = maxDepth(root.right);
        return Math.max(l,r)+1;
    }
}

```

方法二：bfs，层序遍历

```

class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        Queue<TreeNode> queue = new LinkedList<TreeNode>();
        queue.offer(root);
        int ans = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            while (size > 0) {
                TreeNode node = queue.poll();
                if (node.left != null) {
                    queue.offer(node.left);
                }
                if (node.right != null) {
                    queue.offer(node.right);
                }
                size--;
            }
            ans++;
        }
        return ans;
    }
}

```

翻转二叉树

简单

题目：给你一棵二叉树的根节点 **root**，翻转这棵二叉树，并返回其根节点。

解：

先得到左右节点，再交换即可，注意是节点交换，不是值交换。先序遍历和后序遍历都可以，中序不行。

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) {
            return null;
        }
        TreeNode left = invertTree(root.left);
        TreeNode right = invertTree(root.right);
        root.left = right;
        root.right = left;
        return root;
    }
}
```

对称二叉树

简单

题目：给你一个二叉树的根节点 **root**，检查它是否轴对称。

解：

递归，伪代码如下

```
//递归解法
boolean compare(TreeNode left, TreeNode right){ //1. 递归参数
    //2. 终止条件：一个节点为空，一个不为空；都为空；都不为空但值不等
    if() return;
    if() return;
    ...

    //3. 单层递归逻辑
    // 比较外侧
    boolean compareOutside = compare(left.left, right.right);
    // 比较内侧
    boolean compareInside = compare(left.right, right.left);
    return compareOutside && compareInside;
}
```

方法一：递归

```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return check(root.left, root.right);
    }
}
```

```

public boolean check(TreeNode l, TreeNode r) {
    if (l == null && r == null) {
        return true;
    }
    if (l == null && r != null) {
        return false;
    }
    if (l != null && r == null) {
        return false;
    }
    if ((l != null && r != null) && l.val != r.val) {
        return false;
    }
    return check(l.left, r.right) && check(l.right, r.left);
}
}

```

方法二：迭代

```

//迭代解法
public boolean isSymmetric2(TreeNode root) {
    Deque<TreeNode> deque = new LinkedList<>();
    deque.offerFirst(root.left);
    deque.offerLast(root.right);
    while (!deque.isEmpty()) {
        TreeNode leftNode = deque.pollFirst();
        TreeNode rightNode = deque.pollLast();
        if (leftNode == null && rightNode == null) {
            continue;
        }
        // 三个判断条件合并
        if (leftNode == null || rightNode == null || leftNode.val !=
rightNode.val) {
            return false;
        }
        deque.offerFirst(leftNode.left);
        deque.offerFirst(leftNode.right);
        deque.offerLast(rightNode.right);
        deque.offerLast(rightNode.left);
    }
    return true;
}1

```

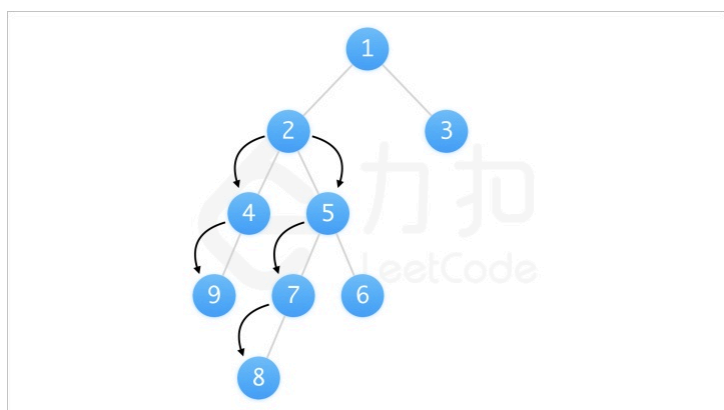
二叉树的直径

简单

题目：给你一棵二叉树的根节点，返回该树的 **直径**。二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度**。这条路径可能经过也可能不经过根节点 **root**。两节点之间路径的 **长度** 由它们之间边数表示。

解：

一条路径的长度为该路径经过的节点数减一，所以求直径（即求路径长度的最大值）等效于求路径经过节点数的最大值减一。任意一条路径均可以被看作由某个节点为起点，从其左儿子和右儿子向下遍历的路径拼接得到。假设我们知道对于该节点的左儿子向下遍历经过最多的节点数 L （即以左儿子为根的子树的深度）和其右儿子向下遍历经过最多的节点数 R （即以右儿子为根的子树的深度），那么以该节点为起点的路径经过节点数的最大值即为 $L+R+1$ 。



```
class Solution {
    int res=1;//节点数量
    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return res-1;//边的数量
    }
    private int dfs(TreeNode root){
        if(root==null) return 0;
        int l = dfs(root.left);
        int r = dfs(root.right);
        res = Math.max(res,l+r+1);//更新路径
        return Math.max(l,r)+1;
    }
}
```

二叉树的层序遍历

中等

题目：给你二叉树的根节点 **root**，返回其节点值的 **层序遍历**。（即逐层地，从左到右访问所有节点）。

解：

方法一：广度优先，迭代法（推荐）

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if(root==null) return res;
        Deque<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        while(!queue.isEmpty()){
            int len = queue.size();//当前层的宽度
            List<Integer> list = new ArrayList<>();
            while(len>0){
                TreeNode node = queue.poll();
                list.add(node.val);
                if(node.left!=null) queue.offer(node.left);
                if(node.right!=null) queue.offer(node.right);
                len--;
            }
            res.add(list);
        }
        return res;
    }
}
```

方法二：深度优先，递归法

```
class Solution {
    List<List<Integer>> list = new ArrayList<>();
    public List<List<Integer>> levelOrder(TreeNode root) {
        dfs(root,0);
        return list;
    }
    public void dfs(TreeNode node,int lever){
        if(node == null) return;
        if(list.size()==lever) list.add(new ArrayList<Integer>());

        list.get(lever).add(node.val);

        dfs(node.left,lever+1);
        dfs(node.right,lever+1);
    }
}
```

将有序数组转换为二叉搜索树

简单——中等

题目：给你一个整数数组 `nums`，其中元素已经按 **升序** 排列，请你将其转换为一棵 **平衡** 二叉搜索树。

解：

二叉搜索树的中序遍历是升序序列，因此题目给的有序数组是二叉搜索树的中序遍历。
注意：给定二叉搜索树的中序遍历，不能唯一地确定二叉搜索树。我们构造一个平衡的二叉树即可（“平衡”是指左右子树的数字个数相同或只相差 1），当节点个数为偶数时，取 $\text{len}/2$ ，或 $\text{len}/2 + 1$ 为中间节点都行，这里取 $\text{len}/2$ 。

尤其注意区间的开闭性，二分的`left`和`right`如何更新

左闭右开：

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return binaryTree(nums, 0, nums.length); //
    }
    //左闭右开
    private TreeNode binaryTree(int[] nums, int l, int r){
        if(l >= r) return null; //
        int mid = l+(r-l)/2;
        TreeNode root = new TreeNode(nums[mid]);
        root.left = binaryTree(nums, l, mid); //
        root.right = binaryTree(nums, mid+1, r);
        return root;
    }
}
```

左闭右闭：

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return binaryTree(nums, 0, nums.length-1); //
    }
    //左闭右闭
    private TreeNode binaryTree(int[] nums, int l, int r){
        if(l > r) return null; //
        int mid = l+(r-l)/2;
        TreeNode root = new TreeNode(nums[mid]);
        root.left = binaryTree(nums, l, mid-1); //
        root.right = binaryTree(nums, mid+1, r);
    }
}
```

```
        return root;
    }
}
```

验证二叉搜索树

中等

题目：给你一个二叉树的根节点 **root**，判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下：

- 节点的左子树只包含小于 当前节点的数。
- 节点的右子树只包含 **大于** 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

解：

方法一：中序遍历，再判断数组是否严格递增即可，略

方法二：更像先序遍历？代码并不好理解。注意min和max在递归遍历左右子树时进行了更新。

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return isEr(root, Long.MIN_VALUE, Long.MAX_VALUE); //初始值
    }
    public static boolean isEr(TreeNode root, long min, long max){
        if(root==null){
            return true;
        }
        long cur = root.val;
        //当前节点，左子树，右子树
        return cur>min && cur<max && isEr(root.left,min,cur) &&
isEr(root.right,cur,max);
    }
}
```

二叉搜索树中第K小的元素

中等

题目：给定一个二叉搜索树的根节点 **root**，和一个整数 **k**，请你设计一个算法查找其中第 **k** 小的元素（从 1 开始计数）。

解：

中序遍历的迭代形式。

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> stack = new ArrayDeque<TreeNode>();
        while (root != null || !stack.isEmpty()) {
            while (root != null) {
                stack.push(root);
                root = root.left;
            }
            root = stack.pop();
            k--;
            if (k == 0) {
                break;
            }
            root = root.right;
        }
        return root.val;
    }
}
```

二叉树的右视图

中等

题目：给定一个二叉树的 **根节点 root**，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

解：

方法一：层序遍历，保存每层最后一个数即可

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Deque<TreeNode> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int len = q.size();
            while(len>0) {
```

```

        TreeNode node = q.poll();
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
        if (len--==1) ans.add(node.val); //先比较再减1
    }
}
return ans;
}
}

```

方法二：dfs

```

class Solution {
    private List<Integer> ans;

    public List<Integer> rightSideView(TreeNode root) {
        ans = new ArrayList<>();
        dfs(root, 0);
        return ans;
    }

    private void dfs(TreeNode node, int depth) {
        if (node == null) return;
        if (ans.size() <= depth)
            ans.add(node.val);
        else
            ans.set(depth, node.val);
        dfs(node.left, depth + 1);
        dfs(node.right, depth + 1);
    }
}

```

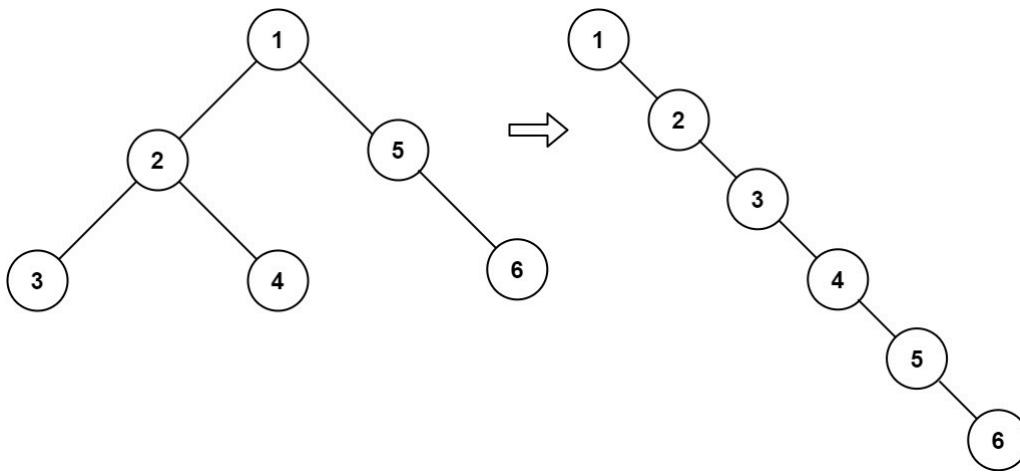
二叉树展开为链表

中等

题目：给你二叉树的根结点 `root`，请你将它展开为一个单链表：

- 展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。
- 展开后的单链表应该与二叉树 **先序遍历** 顺序相同。

示例 1：



输入: root = [1,2,5,3,4,null,6]

输出: [1,null,2,null,3,null,4,null,5,null,6]

解:

方法一: 先序遍历, 再构造元素为TreeNode的链表即可

方法二: 原地操作, 空间复杂度 $O(1)$, 没看见好理解的代码

```
class Solution {
    public void flatten(TreeNode root) {
        List<TreeNode> list = new ArrayList<>();
        dfs(root, list);
        for(int i=1; i<list.size(); i++){
            TreeNode prev = list.get(i-1), cur = list.get(i);
            prev.left = null;
            prev.right = cur;
        }
    }
    public void dfs(TreeNode root, List<TreeNode> list){
        if(root==null) return ;
        list.add(root);
        dfs(root.left, list);
        dfs(root.right, list);
    }
}
```

从前序与中序遍历序列构造二叉树

中等

题目：给定两个整数数组 `preorder` 和 `inorder`，其中 `preorder` 是二叉树的先序遍历，`inorder` 是同一棵树的中序遍历，请构造二叉树并返回其根节点。

解：

原理说明：由先序遍历可以告诉我们中间节点，由中序遍历可以定位到这个中间节点，它的左边就是左子树，右边就是右子树。对左右子树进行这样的递归操作。后序遍历同理

基本步骤：

1. haspmap存储中序遍历元素和位置的对应关系 `map.put(inorder[i], i)`
2. 递归函数 `TreeNode findNode(preorder, 0, preorder.length, inorder, 0, inorder.length)`;
3.
 1. 递归终止条件，左闭右开，`begin >= end`即终止
 2. 找到中间节点，确定左子树节点个数
 3. 递归得到中间节点的`left` 和 `right`节点

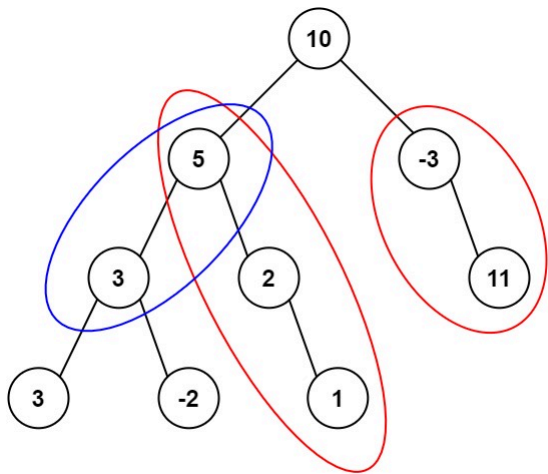
```
class Solution {
    Map<Integer, Integer> map;
    int[] preorder, inorder; // 全局变量简化一下代码
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        // 存储节点值对应的中序位置
        map = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            map.put(inorder[i], i);
        }
        this.preorder = preorder;
        this.inorder = inorder;
        return findNode(0, preorder.length, 0, inorder.length); // 左闭右开
    }

    public TreeNode findNode(int preBegin, int preEnd, int inBegin, int inEnd) {
        if (preBegin >= preEnd || inBegin >= inEnd) return null;
        int rootIndex = map.get(preorder[preBegin]); // 中间节点的位置
        TreeNode root = new TreeNode(inorder[rootIndex]);
        int leftLen = rootIndex - inBegin; // 左子树节点数
        // 注意先序遍历的左右子树分界值
        root.left = findNode(preBegin + 1, preBegin + leftLen + 1, inBegin, rootIndex);
        root.right = findNode(preBegin + leftLen + 1, preEnd, rootIndex + 1, inEnd);
        return root;
    }
}
```

中等

题目：给定一个二叉树的根节点 **root**，和一个整数 **targetSum**，求该二叉树里节点值之和等于 **targetSum** 的 **路径** 的数目。**路径** 不需要从根节点开始，也不需要叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例 1：



输入: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
输出: 3
解释: 和等于 8 的路径有 3 条，如图所示。

解：

方法一：暴力。访问每一个节点 node，检测以 node 为起始节点且向下延深的路径有多少种。递归遍历每一个节点的所有可能的路径，然后将这些路径数目加起来即为返回结果。时间复杂度 $O(N^2)$ ，很慢。

```
class Solution {
    //注意两个方法都在递归
    public int pathSum(TreeNode root, long targetSum) {
        if(root == null) return 0;
        int ret = rootSum(root,targetSum);
        ret+=pathSum(root.left,targetSum);
        ret+=pathSum(root.right,targetSum);
        return ret;
    }
    public int rootSum(TreeNode root, long targetSum){
        int ret = 0;
        if(root==null) return 0;
        long val = (long)root.val;
        if(val==targetSum){
            ret+=1;
        }
        ret+=rootSum(root.left,targetSum-val);
    }
}
```

```

        ret+=rootSum(root.right,targetSum-val);
        return ret;
    }
}

```

方法二：使用前缀和。定义节点的前缀和为：由根结点到当前结点的路径上所有节点的和。

```

class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        //key=和, value=?
        Map<Long, Integer> prefix = new HashMap<Long, Integer>();
        prefix.put(0L, 1);
        return dfs(root, prefix, 0, targetSum);
    }

    public int dfs(TreeNode root, Map<Long, Integer> prefix, long curr, int targetSum) {
        if (root == null) {
            return 0;
        }

        int ret = 0;
        curr += root.val;

        ret = prefix.getOrDefault(curr - targetSum, 0);
        prefix.put(curr, prefix.getOrDefault(curr, 0) + 1);
        ret += dfs(root.left, prefix, curr, targetSum);
        ret += dfs(root.right, prefix, curr, targetSum);
        prefix.put(curr, prefix.getOrDefault(curr, 0) - 1);

        return ret;
    }
}

```

二叉树的最近公共祖先

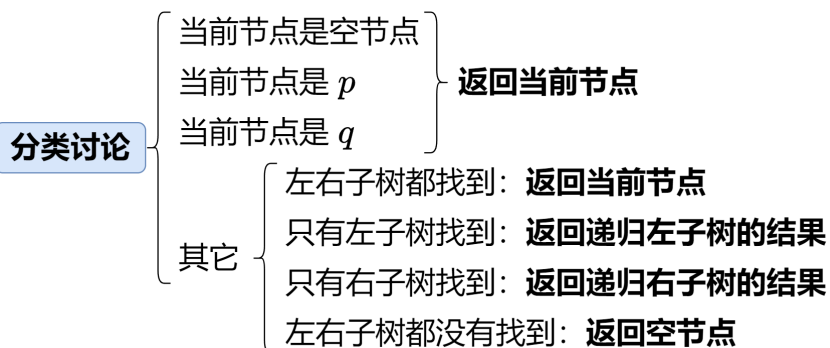
中等

题目：给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。一个节点也可以是它自己的祖先)。”

解：

若 root 是 p,q 的最近公共祖先，则只可能为以下情况之一：

1. p 和 q 在 root 的子树中，且分列 root 的异侧（即分别在左、右子树中）；
2. p=root，且 q 在 root 的左或右子树中；
3. q=root，且 p 在 root 的左或右子树中；



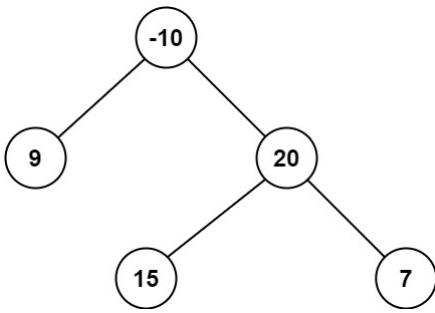
```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        return dfs(root,p,q);
    }
    public TreeNode dfs(TreeNode root, TreeNode p, TreeNode q){
        if(root==null || root==p || root==q) return root;
        TreeNode left = dfs(root.left,p,q); //根节点变为root.left,找到的p和q的最近公共
        祖先
        TreeNode right = dfs(root.right,p,q); //同理
        if(left==null && right==null) return null;
        if(left==null && right!=null) return right;
        if(left!=null && right==null) return left;
        return root;
    }
}
```

二叉树中的最大路径和

困难

题目：二叉树中的 **路径** 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 **至多出现一次**。该路径 **至少包含一个** 节点，且不一定经过根节点。**路径和** 是路径中各节点值的总和。给你一个二叉树的根节点 **root**，返回其 **最大路径和**。

示例 2：



输入: root = [-10,9,20,null,null,15,7]

输出: 42

解释: 最优路径是 15 -> 20 -> 7 , 路径和为 15 + 20 + 7 = 42

解:

代码理解

```
class Solution {
    int maxSum = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        maxGain(root);
        return maxSum;
    }

    public int maxGain(TreeNode node) {
        if (node == null) {
            return 0;
        }
        int leftGain = Math.max(maxGain(node.left), 0); // 左子树的贡献
        int rightGain = Math.max(maxGain(node.right), 0); // 右子树的贡献
        int priceNewpath = node.val + leftGain + rightGain; // 经过该节点的最大路径和
        maxSum = Math.max(maxSum, priceNewpath); // 更新最大路径和
        return node.val + Math.max(leftGain, rightGain); // 从当前节点继续向下的路径的最大贡献, 这个好好理解
    }
}
```

回溯

代码框架

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }
    for (选择: 本层集合中元素 (树中节点孩子的数量就是集合的大小)) {
```



```
        处理节点;  
        backtracking(路径, 选择列表); // 递归  
        回溯, 撤销处理结果  
    }  
}
```

如果是一个集合来求组合的话, 就需要startIndex, 例如: [77.组合](#)(opens new window), [216.组合总和III](#)(opens new window)。

如果是多个集合取组合, 各个集合之间相互不影响, 那么就不用startIndex, 例如: [17.电话号码的字母组合](#)

在求和问题中, 排序之后加剪枝是常见的套路

全排列

中等

题目: 给定一个不含重复数字的数组 `nums`, 返回其 *所有可能的全排列*。你可以 **按任意顺序** 返回答案。

示例 1:

```
输入: nums = [1,2,3]  
输出: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

解:

```
class Solution {  
    //全局属性  
    List<List<Integer>> res = new ArrayList<List<Integer>>(); //存放结果集合  
    List<Integer> path = new ArrayList<>(); //每一个排列  
    boolean[] used;  
  
    public List<List<Integer>> permute(int[] nums) {  
        used = new boolean[nums.length];  
        permuteHelper(nums);  
        return res;  
    }  
  
    private void permuteHelper(int[] nums){  
        if(path.size()==nums.length){  
            res.add(new ArrayList<>(path)); //注意这里是拷贝  
            return;  
        }  
    }  
}
```

```

    }

    for(int i=0;i<nums.length;i++){
        if(!used[i]){//如果还没有用
            //两步
            used[i]=true;
            path.add(nums[i]);
            permuteHelper(nums);//递归
            path.removeLast();//回溯也要两步
            used[i]=false;
        }
    }
}
}
}

```

子集

中等

题目：给你一个整数数组 `nums`，数组中的元素 **互不相同**。返回该数组所有可能的子集（幂集）。解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例 1：

输入：nums = [1,2,3]
 输出：[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

解：

子集是收集树形结构中树的所有节点的结果。而组合问题、分割问题是收集树形结构中叶子节点的结果。

```

class Solution {
    List<List<Integer>> res = new ArrayList<List<Integer>>();//存放结果集合
    List<Integer> path = new ArrayList<>();//每一个排列

    public List<List<Integer>> subsets(int[] nums){
        subsetsHelper(nums,0);
        //可以借助结果理解过程：[[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]
        return res;
    }

    private void subsetsHelper(int[] nums, int startIndex){
        res.add(new ArrayList<>(path));
        for(int i=startIndex;i<nums.length;i++){
            path.add(nums[i]);
        }
    }
}

```

```
        subsetsHelper(nums,i+1);
        path.removeLast();
    }
}
```

电话号码的字母组合

中等

题目：给定一个仅包含数字 **2-9** 的字符串，返回所有它能表示的字母组合。答案可以按**任意顺序** 返回。给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。



示例 1:

```
输入: digits = "23"
输出: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

解:

比较难，结合上面的输出来理解。

```
class Solution {
    List<String> res = new ArrayList<>();//全局
    StringBuilder sb = new StringBuilder();//单个组合
    public List<String> letterCombinations(String digits) {
        if(digits.length()==0){//空
            return res;
        }

        String[] numString = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs",
"tuv", "wxyz"};
        backtracking(digits,numString,0);
        return res;
    }
}
```

```

//num表示组合里面元素个数
private void backtracking(String digits, String[] numString, int num){
    if(num==digits.length()){
        res.add(sb.toString());
        return;
    }

    //字符串中第num位数字字符对应的数字对应的字符串
    String str = numString[digits.charAt(num)-'0'];//"abc"
    for(int i=0;i<str.length();i++){
        sb.append(str.charAt(i));
        backtracking(digits,numString,num+1);
        sb.deleteCharAt(sb.length()-1);//回溯
    }
}
}

```

组合总和

中等

题目：给你一个 **无重复元素** 的整数数组 **candidates** 和一个目标整数 **target**，找出 **candidates** 中可以使数字和为目标数 **target** 的所有 **不同组合**，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。**candidates** 中的 **同一个** 数字可以 **无限制重复被选取**。如果至少一个数字的被选数量不同，则两种组合是不同的。对于给定的输入，保证和为 **target** 的不同组合数少于 **150** 个。

示例 1：

输入：candidates = [2,3,6,7], target = 7
 输出：[[2,2,3],[7]]
 解释：
 2 和 3 可以形成一组候选， $2 + 2 + 3 = 7$ 。注意 2 可以使用多次。
 7 也是一个候选， $7 = 7$ 。
 仅有这两种组合。

解：

```

class Solution {
    List<List<Integer>> res = new ArrayList<>();//全局
    List<Integer> path = new ArrayList<>();//一个解

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);//排序，便于剪枝
    }
}

```

```

        backTracking(candidates,target,0,0);
        return res;
    }

    private void backTracking(int[] candidates, int target,int sum,int idx){
        if(sum==target){
            res.add(new ArrayList<>(path));
            return;
        }

        for(int i=idx; i<candidates.length; i++){
            if(sum+candidates[i]>target){
                break;
            }
            path.add(candidates[i]);
            sum+=candidates[i];
            backTracking(candidates,target,sum,i);//递归
            //回溯
            sum-=candidates[i];
            path.remove(path.size()-1);
        }
    }
}

```

括号生成

中等

题目：数字 n 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且有效的括号组合。

示例 1:

输入: $n = 3$
 输出: ["((()))", "(()())", "(())()", "()(())", "()()()"]

解:

```

class Solution {
    public List<String> generateParenthesis(int n) {
        //n对括号
        List<String> ans = new ArrayList<String>();
        backtrack(ans, new StringBuilder(), 0, 0, n);
        return ans;
    }

    public void backtrack(List<String> ans, StringBuilder cur, int open, int close,

```

```

int max) {
    if (cur.length() == max * 2) {
        ans.add(cur.toString());
        return;
    }
    //如果左括号的数量小于最大值，可以添加一个左括号，并递归调用 backtrack，回溯
    if (open < max) {
        cur.append('(');
        backtrack(ans, cur, open + 1, close, max);
        cur.deleteCharAt(cur.length() - 1);
    }
    //如果右括号的数量小于左括号的数量，可以添加一个右括号，并递归调用 backtrack，回溯
    if (close < open) {
        cur.append(')');
        backtrack(ans, cur, open, close + 1, max);
        cur.deleteCharAt(cur.length() - 1);
    }
}
}

```

单词搜索

中等

题目：给定一个 $m \times n$ 二维字符网格 **board** 和一个字符串单词 **word**。如果 **word** 存在于网格中，返回 **true**；否则，返回 **false**。单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例 1:

A	B	C	E
S	F	C	S
A	D	E	E

输入: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = `"ABCCED"`
 输出: true

解：

```

class Solution {
    public boolean exist(char[][] board, String word) {
        char[] words = word.toCharArray(); // 转为字符数组
        // 遍历搜索的起点
        for(int i = 0; i < board.length; i++) {
            for(int j = 0; j < board[0].length; j++) {
                if (dfs(board, words, i, j, 0)) return true;
            }
        }
        return false;
    }
    // k为当前已经匹配字符的长度
    boolean dfs(char[][] board, char[] word, int i, int j, int k) {
        // 超出边界或者 字符不匹配, 终止
        if (i >= board.length || i < 0 || j >= board[0].length || j < 0 || board[i][j] != word[k]) return false;
        if (k == word.length - 1) return true; // 找到了, 返回true
        // 长度不够但是
        board[i][j] = '\0'; // 标记为空字符, 代表已访问, 很巧妙
        boolean res = dfs(board, word, i + 1, j, k + 1) || dfs(board, word, i - 1, j, k + 1) ||
            dfs(board, word, i, j + 1, k + 1) || dfs(board, word, i, j - 1, k + 1);
        board[i][j] = word[k]; // 回溯
        return res;
    }
}

```

分割回文串

中等

题目：给你一个字符串 **s**，请你将 **s** 分割成一些子串，使每个子串都是 **回文串**。返回 **s** 所有可能的分割方案。

解：

```

// 方法2, 答案的角度
class Solution {
    private final List<List<String>> ans = new ArrayList<>();
    private final List<String> path = new ArrayList<>();
    private String s;

    public List<List<String>> partition(String s) {
        this.s = s;
        dfs(0);
        return ans;
    }
}

```

```

private boolean isPalindrome(int left, int right) { //判断是否是回文子串
    while (left < right)
        if (s.charAt(left++) != s.charAt(right--))
            return false;
    return true;
}

private void dfs(int i) {
    if (i == s.length()) {
        ans.add(new ArrayList<>(path)); // 复制 path
        return;
    }
    for (int j = i; j < s.length(); ++j) { // 枚举子串的结束位置
        if (isPalindrome(i, j)) {
            path.add(s.substring(i, j + 1));
            dfs(j + 1); //判断下一个区间左端
            path.remove(path.size() - 1); // 恢复现场
        }
    }
}
}

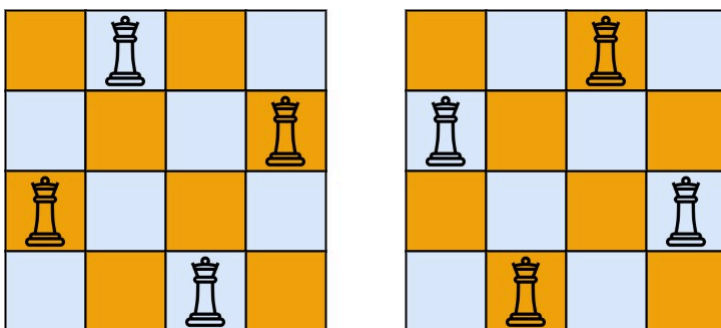
```

N 皇后

困难

题目：按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。**n 皇后问题** 研究的是如何将 **n** 个皇后放置在 **n×n** 的棋盘上，并且使皇后彼此之间不能相互攻击。给你一个整数 **n**，返回所有不同的 **n 皇后问题** 的解决方案。每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 '**Q**' 和 '.' 分别代表了皇后和空位。

示例 1:



输入: **n** = 4

输出: **[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]**

解释：如上图所示，4 皇后问题存在两个不同的解法。

解：

显然，每个皇后必须位于不同行和不同列，因此将 N 个皇后放置在 $N \times N$ 的棋盘上，一定是每一行有且仅有一个皇后，每一列有且仅有一个皇后，且任何两个皇后都不能在同一条斜线上。

由于每个皇后必须位于不同列，因此已经放置的皇后所在的列不能放置别的皇后。第一个皇后有 N 列可以选择，第二个皇后最多有 $N-1$ 列可以选择，第三个皇后最多有 $N-2$ 列可以选择（如果考虑到不能在同一条斜线上，可能的选择数量更少），因此所有可能的情况不会超过 $N!$ 种，遍历这些情况的时间复杂度是 $O(N!)$ 。

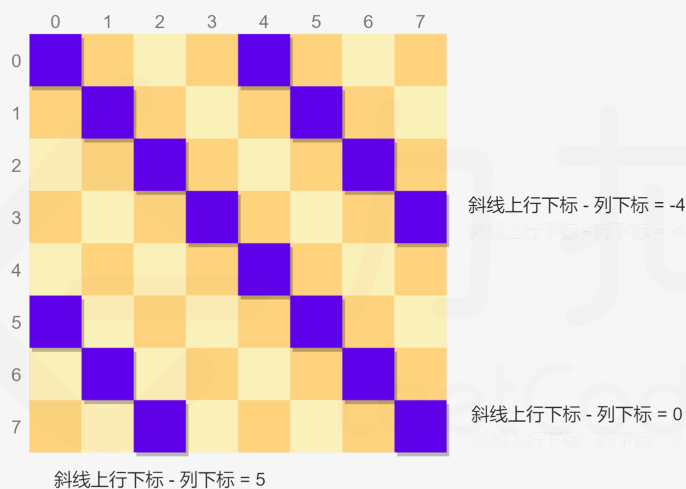
方法一：基于集合的回溯。

一行一行的放置皇后，使用三个集合 *columns*、*diagonals1* 和 *diagonals2* 分别记录每一列以及两个方向（左上-右下，右上-左下）的每条斜线上是否有皇后。

- 列的表示法很直观，一共有 N 列，每一列的下标范围从 0 到 $N-1$ ，使用列的下标即可明确表示每一列。

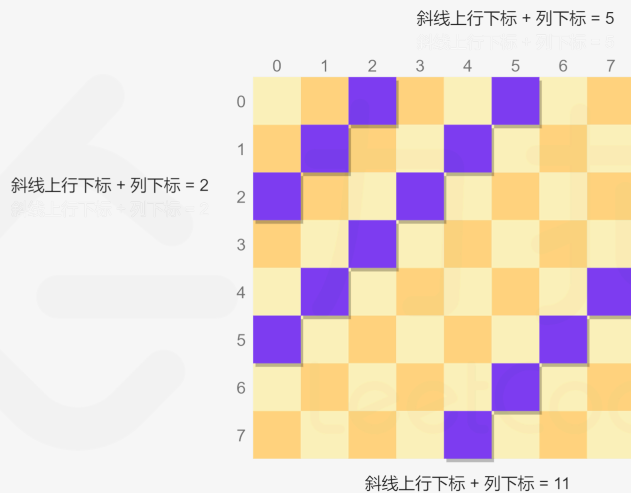
如何表示两个方向的斜线呢？对于每个方向的斜线，需要找到斜线上的每个位置的行下标与列下标之间的关系。

- 方向一的斜线为从左上到右下方向，同一条斜线上的每个位置满足行下标与列下标之差相等，例如 (0,0) 和 (3,3) 在同一条方向一的斜线上。因此使用**行下标与列下标之差**即可明确表示每一条方向一的斜线。



- 方向二的斜线为从右上到左下方向，同一条斜线上的每个位置满足行下标与列下标之和相等，例如 (3,0) 和 (1,2) 在同一条方向二的斜线上。因此使用**行下标与列下标**

之和即可明确表示每一条方向二的斜线



每次放置皇后时，对于每个位置判断其是否在三个集合中，如果三个集合都不包含当前位置，则当前位置是可以放置皇后的位置。

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> solutions = new ArrayList<List<String>>();
        int[] queens = new int[n];
        Arrays.fill(queens, -1);
        Set<Integer> columns = new HashSet<Integer>(); // 列
        Set<Integer> diagonals1 = new HashSet<Integer>(); // 左上一右下
        Set<Integer> diagonals2 = new HashSet<Integer>(); // 右上一左下
        backtrack(solutions, queens, n, 0, columns, diagonals1, diagonals2);
        return solutions;
    }

    public void backtrack(List<List<String>> solutions, int[] queens, int n, int
row, Set<Integer> columns, Set<Integer> diagonals1, Set<Integer> diagonals2) {
        if (row == n) { // 放了n个就收集结果
            List<String> board = generateBoard(queens, n); // 处理成输出格式
            solutions.add(board); // 添加到总的结果里
        } else {
            for (int i = 0; i < n; i++) { // 遍历列
                if (columns.contains(i)) {
                    continue;
                }
                int diagonal1 = row - i;
                if (diagonals1.contains(diagonal1)) {
                    continue;
                }
                int diagonal2 = row + i;
                if (diagonals2.contains(diagonal2)) {
                    continue;
                }
                // 三个方向都不冲突才可以放置
                queens[row] = i;
                // 加入三个冲突集合中
            }
        }
    }
}
```

```

        columns.add(i);
        diagonals1.add(diagonal1);
        diagonals2.add(diagonal2);
        backtrack(solutions, queens, n, row + 1, columns, diagonals1,
diagonals2);//递归
        //回溯
        queens[row] = -1;//标志位
        columns.remove(i);
        diagonals1.remove(diagonal1);
        diagonals2.remove(diagonal2);
    }
}

//转换为指定的输出形式
public List<String> generateBoard(int[] queens, int n) {
    List<String> board = new ArrayList<String>();
    for (int i = 0; i < n; i++) {
        char[] row = new char[n];
        Arrays.fill(row, '.');
        row[queens[i]] = 'Q';
        board.add(new String(row));
    }
    return board;
}
}

```

复杂度分析

- 时间复杂度： $O(N!)$ ，其中 N 是皇后数量。
- 空间复杂度： $O(N)$ ，其中 N 是皇后数量。

栈

有效的括号

简单

题目：给定一个只包括 '('，')'，'{'，'}'，'['，']' 的字符串 s ，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

解：

无数种情形可以总结为三种情况：

- 左括号多了
- 左右括号不匹配
- 右括号多了

```
class Solution {
    public boolean isValid(String s) {
        if (s.length() % 2 != 0) { // 奇数必然不匹配
            return false;
        }
        Stack<Character> stack = new Stack<>(); // 定义一个栈
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') {
                stack.push(')');
            } else if (s.charAt(i) == '[') {
                stack.push(']');
            } else if (s.charAt(i) == '{') {
                stack.push('}');
            } else if (stack.isEmpty()) { // 必须先判断栈是否为空，情况2、3
                return false;
            } else if (stack.pop() != s.charAt(i)) {
                return false;
            }
        }
        return (stack.isEmpty()); // 情况1
    }
}
```

最小栈

中等

题目：设计一个支持 **push**，**pop**，**top** 操作，并能在常数时间内检索到最小元素的栈。

实现 **MinStack** 类：

- **MinStack()** 初始化堆栈对象。
- **void push(int val)** 将元素val推入堆栈。
- **void pop()** 删除堆栈顶部的元素。
- **int top()** 获取堆栈顶部的元素。
- **int getMin()** 获取堆栈中的最小元素。

解：

```
class MinStack {
    //需要用辅助栈，同步记录最小值在栈顶位置
    //1. 主栈压入元素时，辅助栈比较元素和栈顶元素的大小，选择压入新元素还是再压一遍最小
    值；
    //2. 主栈弹出元素时，辅助栈也弹出栈顶元素
    Deque<Integer> stack;
    Deque<Integer> minStack;

    public MinStack() {
        stack = new LinkedList<>();
        minStack = new LinkedList<>();
        minStack.push(Integer.MAX_VALUE);//注意
    }

    public void push(int val) {
        stack.push(val);
        minStack.push(Math.min(minStack.peek(),val));
    }

    public void pop() {
        stack.pop();
        minStack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * int param_3 = obj.top();
 * int param_4 = obj.getMin();
 */
```

字符串解码

中等

题目：给定一个经过编码的字符串，返回它解码后的字符串。

编码规则为: `k[encoded_string]`，表示其中方括号内部的 `encoded_string` 正好重复 `k` 次。注意 `k` 保证为正整数。

你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。

此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 `k`，例如不会出现像 `3a` 或 `2[4]` 的输入。

示例 2:

```
输入: s = "3[a2[c]]"  
输出: "accaccacc"
```

解:

逆序处理], 字母, [, 数字

```
class Solution {  
    public String decodeString(String s) {  
        //模拟，很有难度  
        Stack<Character> stack = new Stack<>();  
        for(char c: s.toCharArray()){  
            if(c != '']){  
                stack.push(c); // 把所有的字母push进去，除了]  
            }  
            else{  
                //1. 取出[]内的字符串  
                StringBuilder sb = new StringBuilder();  
                while(!stack.isEmpty() && Character.isLetter(stack.peek())){  
                    sb.insert(0, stack.pop()); //因为是从栈里面弹出，因此在头部插  
                }  
                String sub = sb.toString();  
                stack.pop(); //去除[  
  
                //2. 获取倍数数字  
                sb = new StringBuilder();  
                while(!stack.isEmpty() && Character.isDigit(stack.peek())){  
                    sb.insert(0, stack.pop()); //因为是从栈里面弹出，因此在头部插  
                }  
                int count = Integer.valueOf(sb.toString()); //倍数  
  
                //3. 根据倍数把字母再push回去  
                while(count > 0){  
                    for(char ch: sub.toCharArray()){  
                        stack.push(ch);  
                    }  
                    count--;  
                }  
            }  
        }  
    }  
}
```

```

    }
}
//把栈里面所有的字母取出来
StringBuilder retv = new StringBuilder();
while(!stack.isEmpty())
    retv.insert(0, stack.pop());

return retv.toString();
}
}

```

每日温度

中等

题目：给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

解：

```

class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        //思路：暴力就两层for循环
        //O(n)，动态规划，逆序
        int n = temperatures.length;
        int[] res = new int[n];
        Deque<Integer> stack = new LinkedList<>(); //小技巧，栈里面存放的是下标
        for(int i=n-1; i>=0; i--){
            int t = temperatures[i];
            while(!stack.isEmpty() && t>temperatures[stack.peek()]){
                stack.pop();
            }
            //如果单调栈还不为空
            if(!stack.isEmpty()){
                res[i] = stack.peek() - i;
            }
            stack.push(i);
        }
        return res;
    }
}

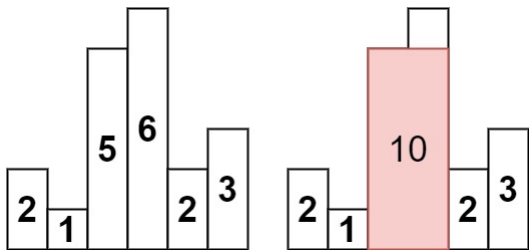
```

柱状图中最大的矩形

困难-没看懂代码

题目：给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:



输入: heights = [2,1,5,6,2,3]
输出: 10
解释: 最大的矩形为图中红色区域，面积为 10

解:

翻译题目：在一维数组中对每一个数找到第一个比自己小的元素。这类“在一维数组中找第一个满足某种条件的数”的场景就是典型的单调栈应用场景。

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        Deque<Integer> stack = new ArrayDeque<>();
        int len = heights.length, res = 0;
        for (int i = 0; i <= len; i++) {
            int h = i == len ? 0 : heights[i];
            if (stack.isEmpty() || h >= heights[stack.peek()]) {
                stack.push(i);
            } else {
                int top = stack.pop();
                res = Math.max(res, heights[top] * (stack.isEmpty() ? i : (i - stack.peek() - 1)));
                i--;
            }
        }
        return res;
    }
}
```

堆

数组中的第K个最大元素

中等

题目：给定整数数组 `nums` 和整数 `k`，请返回数组中第 `k` 个最大的元素。要求时间复杂度为 $O(n)$ 。

解：

桶排序时间复杂度就是 $O(n)$

```
class Solution {
    //O(n)时间复杂度，桶排序的特例，计数排序
    public int findKthLargest(int[] nums, int k) {
        int[] buckets = new int[20001];
        for (int i = 0; i < nums.length; i++) {
            buckets[nums[i] + 10000]++;
        }
        for (int i = 20000; i >= 0; i--) {
            k = k - buckets[i];
            if (k <= 0) {
                return i - 10000;
            }
        }
        return 0;
    }
}
```

前 K 个高频元素

中等

题目：给你一个整数数组 `nums` 和一个整数 `k`，请你返回其中出现频率前 `k` 高的元素。你可以按 **任意顺序** 返回答案。

示例 1:

```
输入：nums = [1,1,1,2,2,3], k = 2
输出：[1,2]
```

解：

优先级队列或自定义比较器

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        //遍历，使用哈希表统计每个数字出现的次数
        Map<Integer, Integer> map = new HashMap<>();
        for(int num:nums){
            map.put(num,map.getOrDefault(num,0)+1);
        }

        //优先级队列，逆序，即大->小
        //但其实自己把map导出为一个二维数组，再写一个比较器，按照第二个维度去排序就是了
        //不要去管什么大堆顶，小堆顶，名词听多了容易糊涂
        PriorityQueue<int[]> pq = new PriorityQueue<>((pair1, pair2) -> pair2[1] - pair1[1]);
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            pq.add(new int[]{entry.getKey(), entry.getValue()}); //服了，这不也是导出成数组了吗
        }

        //结果数组
        int[] ans = new int[k];
        for (int i = 0; i < k; i++) { //依次从队头弹出k个，就是出现频率前k高的元素
            ans[i] = pq.poll()[0];
        }
        return ans;
    }
}
```

数据流的中位数

困难

题目：**中位数**是有序整数列表中的中间值。如果列表的大小是偶数，则没有中间值，中位数是两个中间值的平均值。

- 例如 `arr = [2,3,4]` 的中位数是 `3`。
- 例如 `arr = [2,3]` 的中位数是 $(2 + 3) / 2 = 2.5$ 。

实现 MedianFinder 类:

- `MedianFinder()` 初始化 `MedianFinder` 对象。
- `void addNum(int num)` 将数据流中的整数 `num` 添加到数据结构中。
- `double findMedian()` 返回到目前为止所有元素的中位数。与实际答案相差 `10-5` 以内的答案将被接受。

示例 1:

输入

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

输出

```
[null, null, null, 1.5, null, 2.0]
```

解释

```
MedianFinder medianFinder = new MedianFinder();  
medianFinder.addNum(1);    // arr = [1]  
medianFinder.addNum(2);    // arr = [1, 2]  
medianFinder.findMedian(); // 返回 1.5 ((1 + 2) / 2)  
medianFinder.addNum(3);    // arr[1, 2, 3]  
medianFinder.findMedian(); // return 2.0
```

解:

大小堆一起用来维护中位数。

用两个优先队列 `queMax` 和 `queMin` 分别记录大于中位数的数和小于等于中位数的数。我们尝试添加一个数 `num` 到数据结构中，我们需要分情况讨论，`max{queMin}`表示中位数：

1. $\text{num} \leq \max\{\text{queMin}\}$

此时 `num` 小于等于中位数，我们需要将该数添加到 `queMin` 中。新的中位数将小于等于原来的中位数，因此我们可能需要将 `queMin` 中最大的数移动到 `queMax` 中。

2. $\text{num} > \max\{\text{queMin}\}$

此时 `num` 大于中位数，我们需要将该数添加到 `queMin` 中。新的中位数将大于等于原来的中位数，因此我们可能需要将 `queMax` 中最小的数移动到 `queMin` 中。

```
class MedianFinder {  
    PriorityQueue<Integer> left = new PriorityQueue<>((a,b)->b-a);  
    PriorityQueue<Integer> right = new PriorityQueue<>((a,b)->a-b);  
  
    public MedianFinder() {  
    }  
  
    public void addNum(int num) {  
        int l = left.size(), r=right.size();  
        if(l==r){  
            if(right.isEmpty() || num<=right.peek()){  
                left.add(num);  
            }  
        }  
    }  
}
```

```

        }else{
            left.add(right.poll());
            right.add(num);
        }
    }else{
        if(num>=left.peek()){
            right.add(num);
        }else{
            right.add(left.poll());
            left.add(num);
        }
    }
}

public double findMedian() {
    if(left.size()==right.size()){
        return (right.peek()+left.peek())/2.0;
    }else{
        return left.peek();
    }
}
}

```

```

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */

```